

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

# Scan ID 3	Date April 13, 2026
Organization haycarlitos	Repository shhh-wallet-cairo
Branch main	Commit Hash 70eeef3c...f83ed1d4

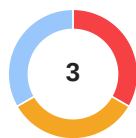
Contracts in scope

src/ed25519/component.cairo src/ed25519/interface.cairo src/lib.cairo src/outside_execution.cairo
src/wallet.cairo tests/test_contract.cairo

Code Statistics

Findings 3	Contracts Scanned 6	Lines of Code 730
---------------	------------------------	----------------------

Findings Summary



Total Findings

- High Risk (1)
- Medium Risk (1)
- Low Risk (0)
- Info (1)
- Best Practices (0)

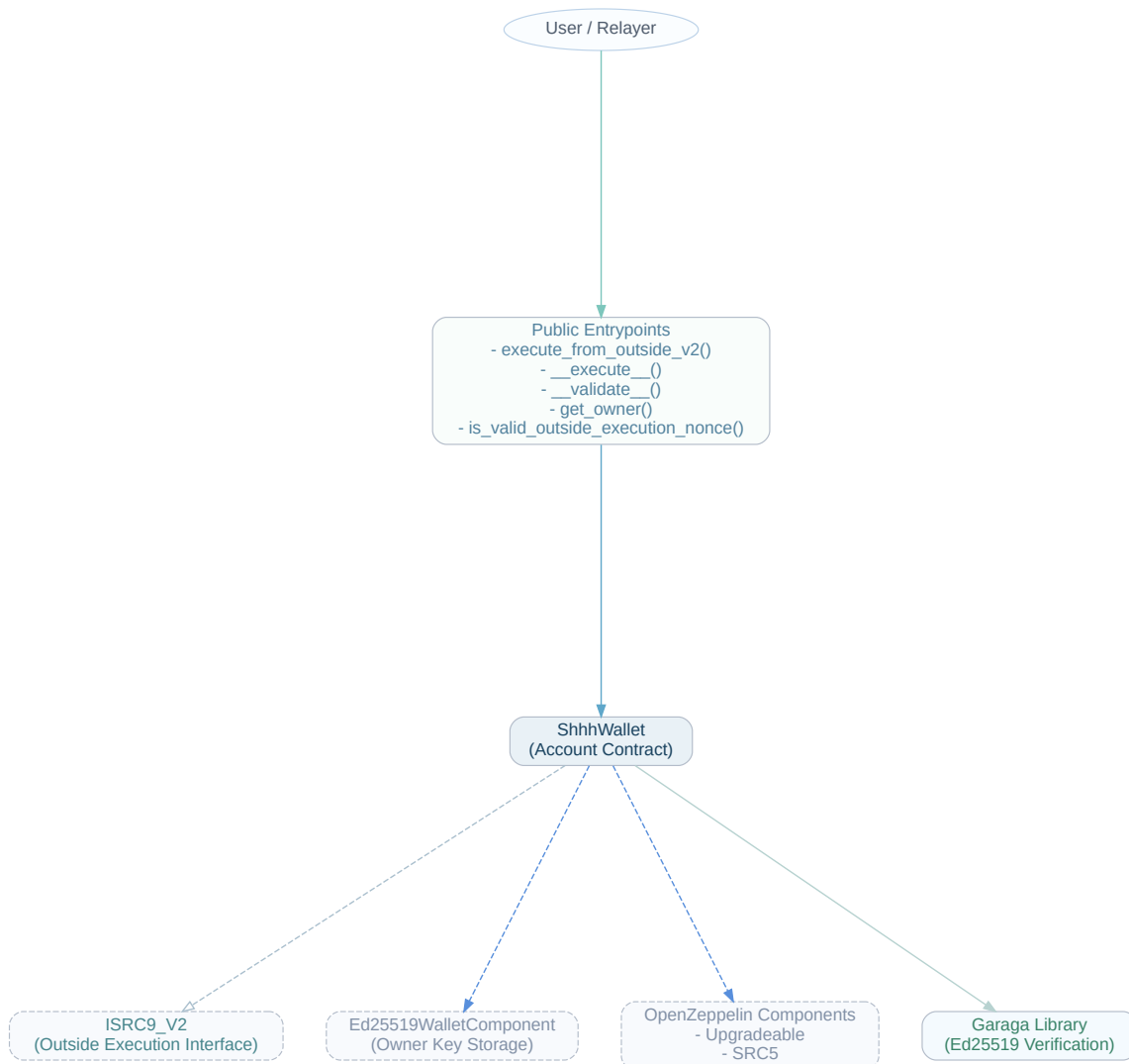
Code Summary

The protocol is a specialized Starknet account contract designed to authenticate transactions using Ed25519 signatures rather than the native Stark curve. It acts as a smart contract wallet that strictly enforces meta-transactions via the SNIP-9 (Outside Execution v2) standard. By disabling standard direct invocations, the wallet ensures that all state-modifying operations are executed as sponsored transactions or relayed payloads. The architecture leverages the Garaga cryptographic library to verify EdDSA signatures and includes custom byte-to-hex-ASCII encoding to maintain compatibility with Phantom wallet signature formats. This allows users to control a Starknet account using an external Ed25519 keypair seamlessly.

Main entry points and actors:

- `__execute__`: A standard Starknet account multicall entry point used primarily for transaction fee estimation, processing an array of calls. Actors: Starknet OS or Bundlers.
- `execute_from_outside_v2`: The primary execution engine of the wallet that processes SNIP-9 outside execution payloads, verifies the Ed25519 signature, consumes the nonce, and executes the requested calls. Actors: Relayers, Paymasters, or designated external callers.

Code Diagram



✦ 1 of 3 Findings

src/wallet.cairo

Unrestricted `__execute__` lets any Starknet account execute arbitrary calls from the wallet

• High Risk

The account exposes a public `__execute__` entrypoint that performs arbitrary multicalls but does not authenticate the caller at all:

```
#[external(v0)]
fn __execute__(ref self: ContractState, calls: Array<Call>) -> Array<Span<felt252>> {
    let mut results: Array<Span<felt252>> = array![];
    for call in calls {
        match syscalls::call_contract_syscall(call.to, call.selector, call.calldata) {
            Result::Ok(ret) => results.append(ret),
            Result::Err(_) => results.append(array![]span()),
        }
    };
    results
}
```

This is a full authorization bypass. On Starknet, `__validate__` is only run when this contract itself is used as the transaction sender account. It is **not** run when some other account simply invokes `ShhhWallet::__execute__` as a normal external function. Because `__execute__` has no caller check and no Ed25519 verification, any arbitrary Starknet account can call it directly and make the wallet call other contracts as itself.

That allows an attacker to spend any STRK/ERC20/NFTs held by the wallet, grant token approvals, interact with DeFi positions, or trigger any other action available to the wallet contract address. This directly contradicts the stated security model that execution authority is exclusively derived from `execute_from_outside_v2` plus Ed25519 signature verification.

✦ 2 of 3 Findings

src/wallet.cairo

Authorized multicalls are non-atomic and silently ignore failed subcalls while still consuming the signed nonce

• Medium Risk

Both execution paths swallow subcall failures instead of reverting the overall operation. In `execute_from_outside_v2`, the nonce is marked as used before any external calls are dispatched, and each failed subcall is converted into an empty return span:

```
assert(!self.outside_nonces.read(outside_execution.nonce), 'SRC9: duplicate nonce');
self.outside_nonces.write(outside_execution.nonce, true);
...
match syscalls::call_contract_syscall(*call.to, *call.selector, *call.calldata) {
  Result::Ok(ret) => results.append(ret),
  Result::Err(_) => results.append(array![].span()),
}
```

Because the function does not revert on downstream failure, a signed outside-execution payload can be only partially applied. Earlier calls may succeed, later calls may fail, and the wallet still returns successfully with the nonce permanently consumed. This is materially different from the atomic behavior users and integrators normally expect from account multicalls.

The behavior is exploitable in the wallet's meta-transaction model because the signed payload is typically executable by `ANY_CALLER`. Any relayer or observer that obtains a valid signed order can choose when to submit it inside the allowed time window. They can wait for a state where one of the calls is expected to fail while other calls still succeed, forcing unintended partial effects under the wallet's authority and burning the one-time nonce so the user cannot replay the same signed payload once conditions recover. Workflows that assume all-or-nothing semantics, such as chained swaps, conditional payments, debt-management sequences, or multi-step approval/spend flows, can therefore end in a materially different final state than the signer intended.

Severity Note:

- Users frequently set caller to `ANY_CALLER` to allow third-party relaying.
- Signed multicall sequences include steps where partial success can be economically harmful (e.g., multi-leg swaps, approval-then-spend flows).
- An external relayer can anticipate conditions under which one or more later subcalls will revert while earlier steps still succeed.

✦ 3 of 3 Findings

src/wallet.cairo

Contract is not upgradeable due to missing external upgrade function[• Info](#)

The `ShhhWallet` contract imports the OpenZeppelin `UpgradeableComponent` and is intended to be upgradeable. However, it fails to expose an external function (e.g., `upgrade`) that utilizes the internal `_upgrade` method. Furthermore, since `execute_from_outside_v2` only allows calling other contracts via `call_contract_syscall` and cannot directly invoke syscalls like `replace_class_syscall`, there is no mechanism for the wallet owner to upgrade the contract. The upgradeability feature is effectively dead code, permanently locking the contract to its initial implementation.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.